

Laboratorio 4

Desarrollo Backend Cloud-Native con AWS Lambda y DynamoDB

Nombre de Integrantes del Equipo:

1.	
2.	
3.	
4.	
5.	
6.	

Tecnologías utilizadas

- Amazon Web Services
- Terraform
- Python
- AWS Lambda
- DynamoDB
- API Gateway
- CloudWatch

Objetivo

Desarrollar un backend serverless funcional utilizando:

- Terraform,
- AWS Lambda,
- DynamoDB,
- API Gateway,
- y Python.

El laboratorio está orientado a:

- desarrollo backend,
- integración cloud,
- debugging,
- manejo de errores,
- y resolución de problemas reales.

NO se permite

- crear recursos manualmente desde AWS Console,
- modificar recursos desde la consola,
- crear tablas manualmente,
- crear Lambdas manualmente,
- crear APIs manualmente.

TODO debe ser creado usando Terraform.

Evidencias

Las siguientes evidencias deben colocarse al inicio del documento entregable.

Evidencia 1

Captura de:

terraform apply

mostrando:

- tablas DynamoDB,
- Lambdas,
- API Gateway,
- IAM Roles,
- y permisos creados exitosamente.

Evidencia 2

Captura del endpoint funcionando desde:

- Postman,
- curl,
- o navegador.

Debe mostrar:

- respuesta HTTP 200,
- y JSON retornado.

Evidencia 3

Captura de registros almacenados en DynamoDB.

Debe visualizarse:

- cliente creado,
- id,
- correo,
- nombre.

Evidencia 4

Captura de logs exitosos en CloudWatch.

Debe visualizarse:

- ejecución Lambda,
- logs personalizados,
- request procesado.

Evidencia 5

Captura del proyecto completo.

Debe mostrar:

- estructura carpetas,
- archivos `.tf`,
- Lambdas implementadas.

Escenario

Una institución bancaria necesita un sistema serverless que permita:

- registrar clientes,
- consultar cuentas,
- realizar transferencias,
- validar saldo,
- almacenar historial de movimientos.

Todo debe implementarse usando:

- Lambda,
- DynamoDB,
- API Gateway,
- y Terraform.

Parte I — Estructura del proyecto

Paso 1

Crear estructura:

```
laboratorio4/
├── terraform/
│   ├── provider.tf
│   ├── variables.tf
│   ├── main.tf
│   └── outputs.tf
├── lambdas/
│   ├── crear_cliente/
│   │   ├── main.py
│   │   ├── requirements.txt
│   │   └── lambda.zip
│   └── realizar_transferencia/
│       ├── main.py
│       ├── requirements.txt
│       └── lambda.zip
└── README.md
```

Parte II — Configuración Terraform

Paso 2

Crear archivo provider.

Paso 3

Agregar la configuración de la región al archivo provider.

Paso 4

Crear archivo variables.tf

Paso 5

Agregar variable de región de AWS.

Parte III — DynamoDB

Paso 6

Abrir archivo:

terraform/main.tf

Paso 7

Copiar y pegar:

```
resource "aws_dynamodb_table" "clientes" {

  name      = "clientes"
  billing_mode = "PAY_PER_REQUEST"

  hash_key = "id"

  attribute {
    name = "id"
    type = "S"
  }
}

resource "aws_dynamodb_table" "cuentas" {

  name      = "cuentas"
  billing_mode = "PAY_PER_REQUEST"

  hash_key = "cuenta_id"

  attribute {
    name = "cuenta_id"
    type = "S"
  }
}

resource "aws_dynamodb_table" "transferencias" {

  name      = "transferencias"
  billing_mode = "PAY_PER_REQUEST"
```

```
hash_key = "transferencia_id"

attribute {
  name = "transferencia_id"
  type = "S"
}
}
```

Parte IV — Lambda Crear Cliente

Paso 8

Crear archivo:

lambdas/crear_cliente/main.py

Paso 9

Copiar y pegar:

```
import json
import boto3
import uuid
from datetime import datetime

dynamodb = boto3.resource('dynamodb')

tabla = dynamodb.Table('clientes')

def lambda_handler(event, context):

    try:

        print("===== NUEVO REQUEST =====")

        body = json.loads(event['body'])

        if 'nombre' not in body:
            return {
                'statusCode': 400,
                'body': json.dumps({
                    'mensaje': 'El nombre es obligatorio'
                })
            }
    }
```

```
if 'correo' not in body:
    return {
        'statusCode': 400,
        'body': json.dumps({
            'mensaje': 'El correo es obligatorio'
        })
    }
```

```
if body['nombre'] == "":
    return {
        'statusCode': 400,
        'body': json.dumps({
            'mensaje': 'Nombre vacío'
        })
    }
```

```
if body['correo'] == "":
    return {
        'statusCode': 400,
        'body': json.dumps({
            'mensaje': 'Correo vacío'
        })
    }
```

```
cliente = {
    "id": str(uuid.uuid4()),
    "nombre": body['nombre'],
    "correo": body['correo'],
    "fecha_creacion": datetime.utcnow().isoformat()
}
```

```
tabla.put_item(Item=cliente)
```

```
print(f"Cliente almacenado: {cliente}")
```

```
return {
    'statusCode': 200,
    'body': json.dumps(cliente)
}
```

```
except Exception as e:
```

```
    print(f"ERROR: {str(e)}")
```

```
    return {
        'statusCode': 500,
        'body': json.dumps({
```

```
        'mensaje': 'Error interno'
    })
}
```

Paso 10

Crear archivo:

lambdas/crear_cliente/requirements.txt

Paso 11

Agregar al archivo requirements.txt el siguiente texto:

boto3

Paso 12

Lee la teoría sobre boto3:

boto3 es el SDK oficial de Python para Amazon Web Services (AWS).

Permite que programas escritos en Python puedan comunicarse con los servicios de AWS mediante código.

Con boto3 puedes:

- leer y escribir en DynamoDB,
- subir archivos a S3,
- crear máquinas EC2,
- enviar mensajes a SQS,
- ejecutar Lambdas,
- consultar CloudWatch,
- y administrar recursos AWS automáticamente.

Paso 13

Desde:

lambdas/crear_cliente

ejecutar:

zip -r lambda.zip .

Nota importante

Si utiliza Windows y aparece:

zip is not recognized

utilizar:

- Git Bash,
- WSL,
- o terminal Linux integrada en VSCode.

Parte V — Lambda Transferencias

Paso 14

Crear archivo:

lambdas/realizar_transferencia/main.py

Paso 15

Copiar y pegar:

```
import json
import boto3
import uuid
from datetime import datetime

dynamodb = boto3.resource('dynamodb')

tabla = dynamodb.Table('transferencias')

def lambda_handler(event, context):

    try:

        print("===== NUEVA TRANSFERENCIA =====")

        body = json.loads(event['body'])

        transferencia = {
            "transferencia_id": str(uuid.uuid4()),
            "cuenta_origen": body['cuenta_origen'],
            "cuenta_destino": body['cuenta_destino'],
```

```

        "monto": body['monto'],
        "fecha_transferencia": datetime.utcnow().isoformat()
    }

    tabla.put_item(Item=transferencia)

    print(f"Transferencia almacenada: {transferencia}")

    return {
        'statusCode': 200,
        'body': json.dumps(transferencia)
    }

except Exception as e:

    print(f"ERROR: {str(e)}")

    return {
        'statusCode': 500,
        'body': json.dumps({
            'mensaje': 'Error en transferencia'
        })
    }

```

Paso 16

Crear archivo:

lambdas/realizar_transferencia/requirements.txt

Paso 17

Agregar:

boto3

Paso 18

Desde:

lambdas/realizar_transferencia

ejecutar:

zip -r lambda.zip .

Parte VI — IAM + Lambda + API Gateway

Paso 19

Abrir archivo:

terraform/main.tf

Paso 20

Copiar y pegar:

```
resource "aws_iam_role" "lambda_role" {

  name = "lambda_role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"

    Statement = [{
      Action = "sts:AssumeRole"

      Effect = "Allow"

      Principal = {
        Service = "lambda.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "lambda_logs" {

  role      = aws_iam_role.lambda_role.name

  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}

resource "aws_iam_role_policy" "dynamodb_policy" {

  name = "dynamodb_policy"

  role = aws_iam_role.lambda_role.id

  policy = jsonencode({
```

```
Version = "2012-10-17"
```

```
Statement = [{  
    Effect = "Allow"
```

```
    Action = [  
        "dynamodb:*"  
    ]
```

```
    Resource = "*"
}]
})
}
```

```
resource "aws_lambda_function" "crear_cliente" {
```

```
    function_name = "crear_cliente"
```

```
    runtime = "python3.11"
```

```
    handler = "main.lambda_handler"
```

```
    filename = "../lambdas/crear_cliente/lambda.zip"
```

```
    source_code_hash = filebase64sha256("../lambdas/crear_cliente/lambda.zip")
```

```
    role = aws_iam_role.lambda_role.arn
}
```

```
resource "aws_lambda_function" "realizar_transferencia" {
```

```
    function_name = "realizar_transferencia"
```

```
    runtime = "python3.11"
```

```
    handler = "main.lambda_handler"
```

```
    filename = "../lambdas/realizar_transferencia/lambda.zip"
```

```
    source_code_hash = filebase64sha256("../lambdas/realizar_transferencia/lambda.zip")
```

```
    role = aws_iam_role.lambda_role.arn
}
```

```
resource "aws_api_gateway_rest_api" "api" {
```

```
    name = "laboratorio-api"
}
```

Parte VII — API Gateway Clientes

Paso 21

Copiar y pegar:

```
resource "aws_api_gateway_resource" "clientes_resource" {  
  
  rest_api_id = aws_api_gateway_rest_api.api.id  
  
  parent_id = aws_api_gateway_rest_api.api.root_resource_id  
  
  path_part = "clientes"  
}
```

Paso 22

Copiar y pegar:

```
resource "aws_api_gateway_method" "clientes_post" {  
  
  rest_api_id = aws_api_gateway_rest_api.api.id  
  
  resource_id = aws_api_gateway_resource.clientes_resource.id  
  
  http_method = "POST"  
  
  authorization = "NONE"  
}
```

Paso 23

Copiar y pegar:

```
resource "aws_api_gateway_integration" "clientes_integration" {  
  
  rest_api_id = aws_api_gateway_rest_api.api.id  
  
  resource_id = aws_api_gateway_resource.clientes_resource.id  
  
  http_method = aws_api_gateway_method.clientes_post.http_method  
  
  integration_http_method = "POST"
```

```
type = "AWS_PROXY"

uri = aws_lambda_function.crear_cliente.invoke_arn
}
```

Paso 24

Copiar y pegar:

```
resource "aws_lambda_permission" "api_gateway_permission" {

  statement_id = "AllowAPIGatewayInvoke"

  action = "lambda:InvokeFunction"

  function_name = aws_lambda_function.crear_cliente.function_name

  principal = "apigateway.amazonaws.com"

  source_arn = "${aws_api_gateway_rest_api.api.execution_arn}/*/*"
}
```

Parte VIII — API Gateway Transferencias

Paso 25

Copiar y pegar:

```
resource "aws_api_gateway_resource" "transferencias_resource" {

  rest_api_id = aws_api_gateway_rest_api.api.id

  parent_id = aws_api_gateway_rest_api.api.root_resource_id

  path_part = "transferencias"
}
```

Paso 26

Copiar y pegar:

```
resource "aws_api_gateway_method" "transferencias_post" {

  rest_api_id = aws_api_gateway_rest_api.api.id
```

```
resource_id = aws_api_gateway_resource.transferencias_resource.id

http_method = "POST"

authorization = "NONE"
}
```

Paso 27

Copiar y pegar:

```
resource "aws_api_gateway_integration" "transferencias_integration" {

  rest_api_id = aws_api_gateway_rest_api.api.id

  resource_id = aws_api_gateway_resource.transferencias_resource.id

  http_method = aws_api_gateway_method.transferencias_post.http_method

  integration_http_method = "POST"

  type = "AWS_PROXY"

  uri = aws_lambda_function.realizar_transferencia.invoke_arn
}
```

Paso 28

Copiar y pegar:

```
resource "aws_lambda_permission" "transferencias_permission" {

  statement_id = "AllowTransferenciasInvoke"

  action = "lambda:InvokeFunction"

  function_name = aws_lambda_function.realizar_transferencia.function_name

  principal = "apigateway.amazonaws.com"

  source_arn = "${aws_api_gateway_rest_api.api.execution_arn}/*/*"
}
```

Paso 29

Copiar y pegar:

```
resource "aws_api_gateway_deployment" "api_deployment" {

  depends_on = [
    aws_api_gateway_integration.clientes_integration,
    aws_api_gateway_integration.transferencias_integration
  ]

  triggers = {
    redeployment = sha1(jsonencode([
      aws_api_gateway_resource.clientes_resource.id,
      aws_api_gateway_resource.transferencias_resource.id
    ]))
  }

  rest_api_id = aws_api_gateway_rest_api.api.id

  stage_name = "dev"
}
```

Parte IX — Outputs Terraform

Paso 30

Crear archivo:

terraform/outputs.tf

Paso 31

Copiar y pegar:

```
output "clientes_url" {

  value = "${aws_api_gateway_deployment.api_deployment.invoke_url}/clientes"
}

output "transferencias_url" {

  value = "${aws_api_gateway_deployment.api_deployment.invoke_url}/transferencias"
}
```

Parte X — Despliegue

Paso 32

Desde:

terraform/

ejecutar:

terraform init

Paso 33

Ejecutar:

terraform fmt

Paso 34

Ejecutar:

terraform apply

Paso 35

Escribir:

yes

cuando Terraform lo solicite.

A screenshot of a terminal window with a dark background. The terminal shows the output of a Terraform apply command. It lists several actions to be performed, including 'lambda:InvokeFunction' and 'apigateway:amazonaws.com'. It also shows the plan for adding, changing, or destroying resources. The terminal prompts the user to confirm the actions, and the user has entered 'yes'.

```
TERMINAL PROBLEMS OUTPUT PORTS DEBUG CONSOLE  
+ action = "lambda:InvokeFunction"  
+ function_name = "realizar_transferencia"  
+ id = (known after apply)  
+ principal = "apigateway.amazonaws.com"  
+ region = "us-east-1"  
+ source_arn = (known after apply)  
+ statement_id = "Allow(transferenciasInvoke)"  
+ statement_id_prefix = (known after apply)  
}  
  
Plan: 19 to add, 0 to change, 0 to destroy.  
  
Changes to Outputs:  
+ clientes_url = (known after apply)  
+ transferencias_url = (known after apply)  
  
Do you want to perform these actions?  
Terraform will perform the actions described above.  
Only 'yes' will be accepted to approve.  
  
Enter a value: yes
```

```
TERMINAL  PROBLEMS  OUTPUT  PORTS  DEBUG CONSOLE
aws_lambda_permission.transferencias_permission: Creation complete after 0s [id=AllowTransferenciasInvoke]
aws_dynamodb_table.transferencias: Creation complete after 18s [id=transferencias]
aws_dynamodb_table.cuentas: Creation complete after 18s [id=cuentas]
aws_lambda_function.crear_cliente: Still creating... [00m20s elapsed]
aws_lambda_function.crear_cliente: Creation complete after 22s [id=crear_cliente]
aws_api_gateway_integration.clientes_integration: Creating...
aws_lambda_permission.api_gateway_permission: Creating...
aws_api_gateway_integration.clientes_integration: Creation complete after 1s [id=agi-z95c8ow2y5-0mq3hv-POST]
aws_api_gateway_deployment.api_deployment: Creating...
aws_lambda_permission.api_gateway_permission: Creation complete after 1s [id=AllowAPIGatewayInvoke]
aws_api_gateway_deployment.api_deployment: Creation complete after 0s [id=lr2f14]
aws_api_gateway_stage.dev: Creating...
aws_api_gateway_stage.dev: Creation complete after 1s [id=ags-z95c8ow2y5-dev]

Apply complete! Resources: 19 added, 0 changed, 0 destroyed.

Outputs:

clientes_url = "https://z95c8ow2y5.execute-api.us-east-1.amazonaws.com/dev/clientes"
transferencias_url = "https://z95c8ow2y5.execute-api.us-east-1.amazonaws.com/dev/transferencias"
```

Paso 36

Copiar output:

clientes_url

clientes_url = "https://y5vlxj0rhc.execute-api.us-east-1.amazonaws.com/dev/clientes"

transferencias_url = <https://y5vlxj0rhc.execute-api.us-east-1.amazonaws.com/dev/transferencias>

Paso 37

Copiar output:

transferencias_url

Parte XI — Pruebas Clientes

Paso 38

Abrir Postman.

Paso 39

Seleccionar:

POST

Paso 40

Seleccionar:

Body → raw → JSON

Paso 41

Usar URL:

clientes_url

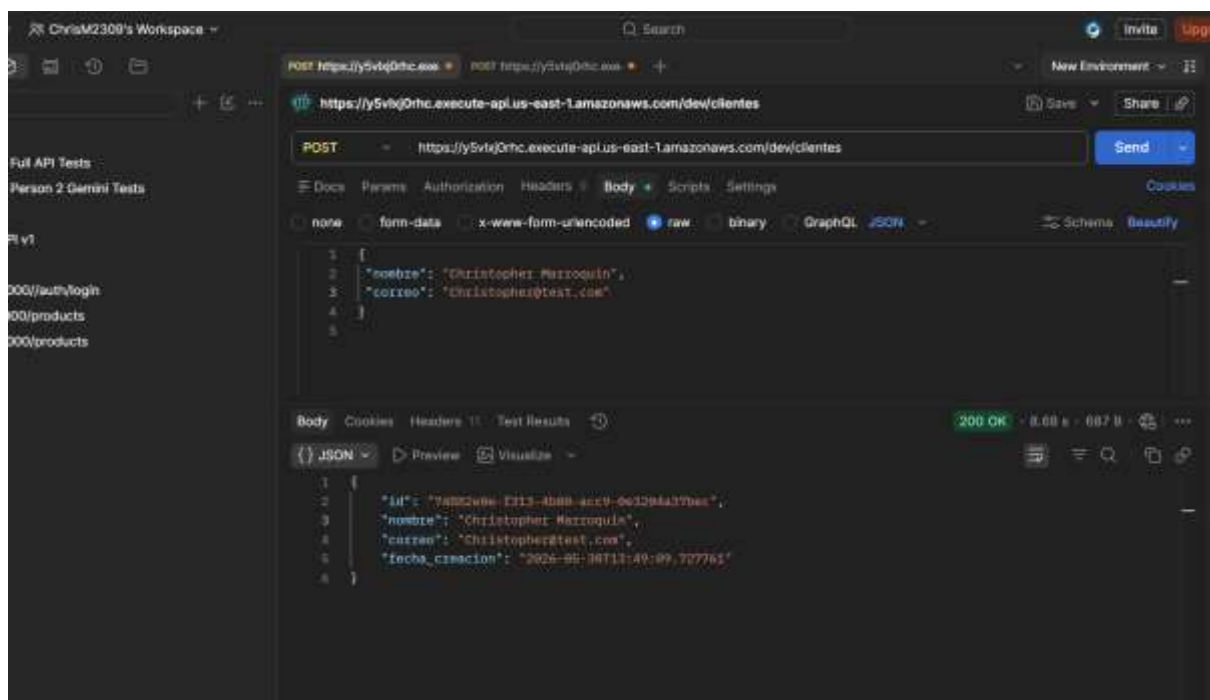
Paso 42

Enviar:

```
{  
  "nombre": "Luz Maria",  
  "correo": "luz@test.com"  
}
```

Paso 43

Validar respuesta HTTP 200.



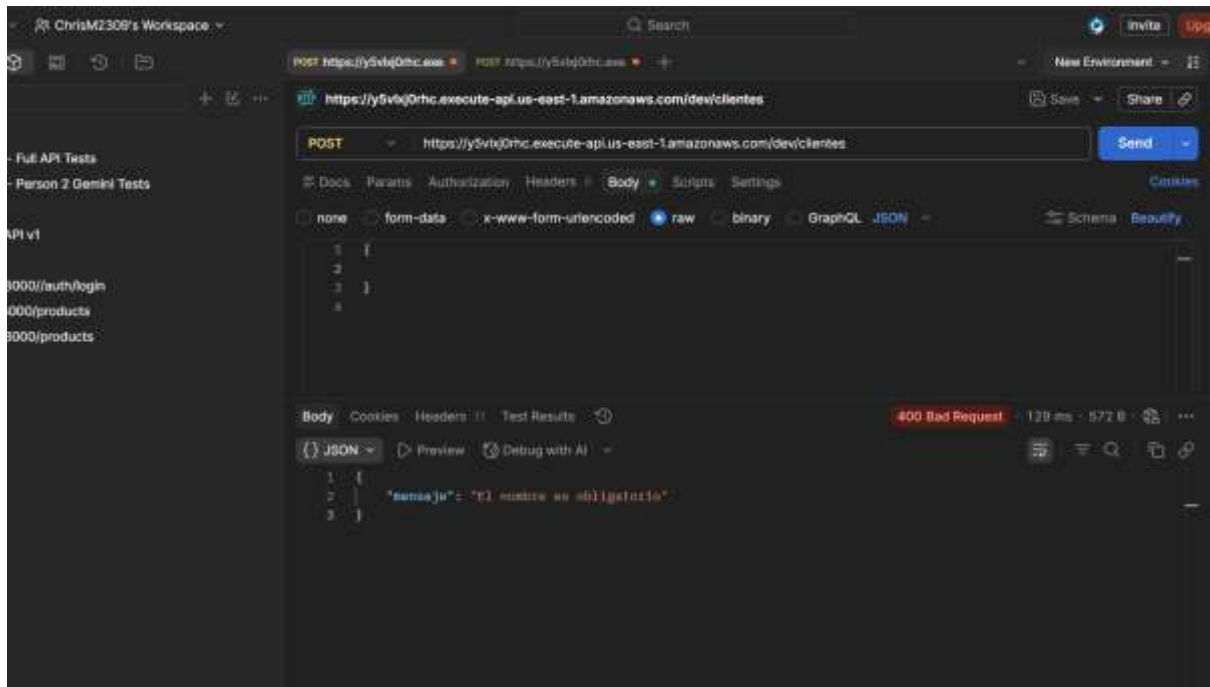
Paso 44

Enviar:

```
{
```

Paso 45

Validar respuesta HTTP 400.



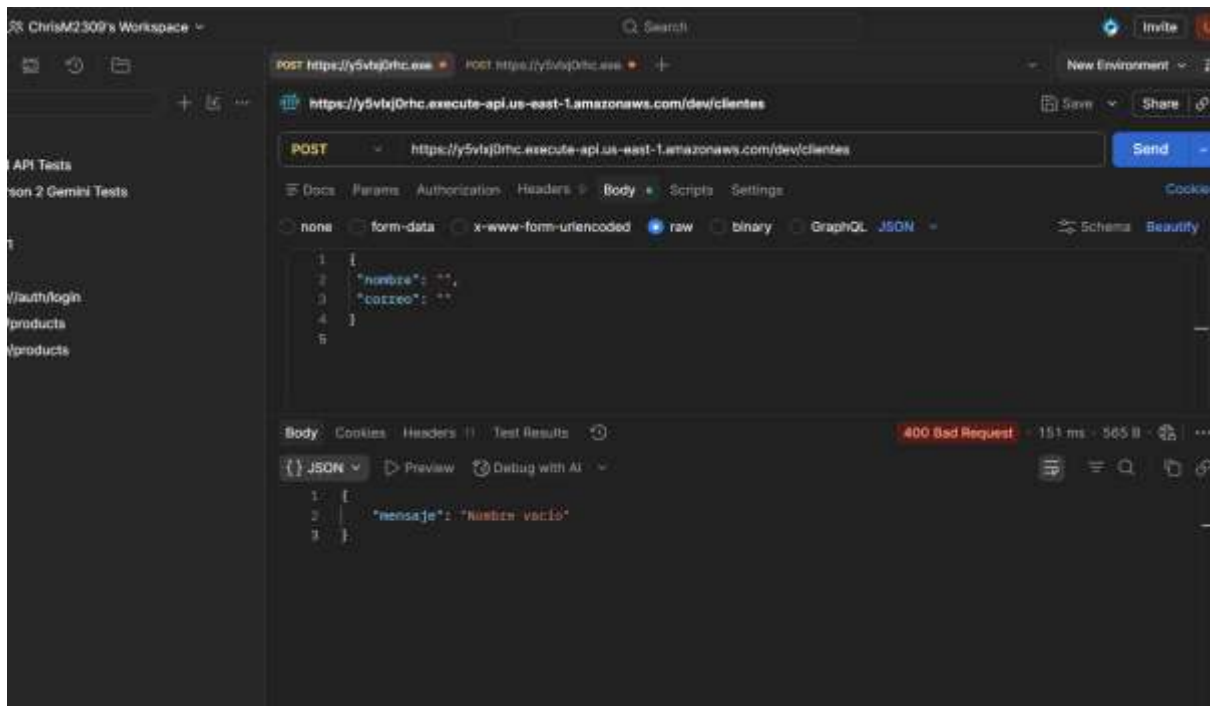
Paso 46

Enviar:

```
{  
  "nombre": "",  
  "correo": ""  
}
```

Paso 47

Validar respuesta HTTP 400.



Parte XII — Pruebas Transferencias

Paso 48

Usar URL:

`transferencias_url`

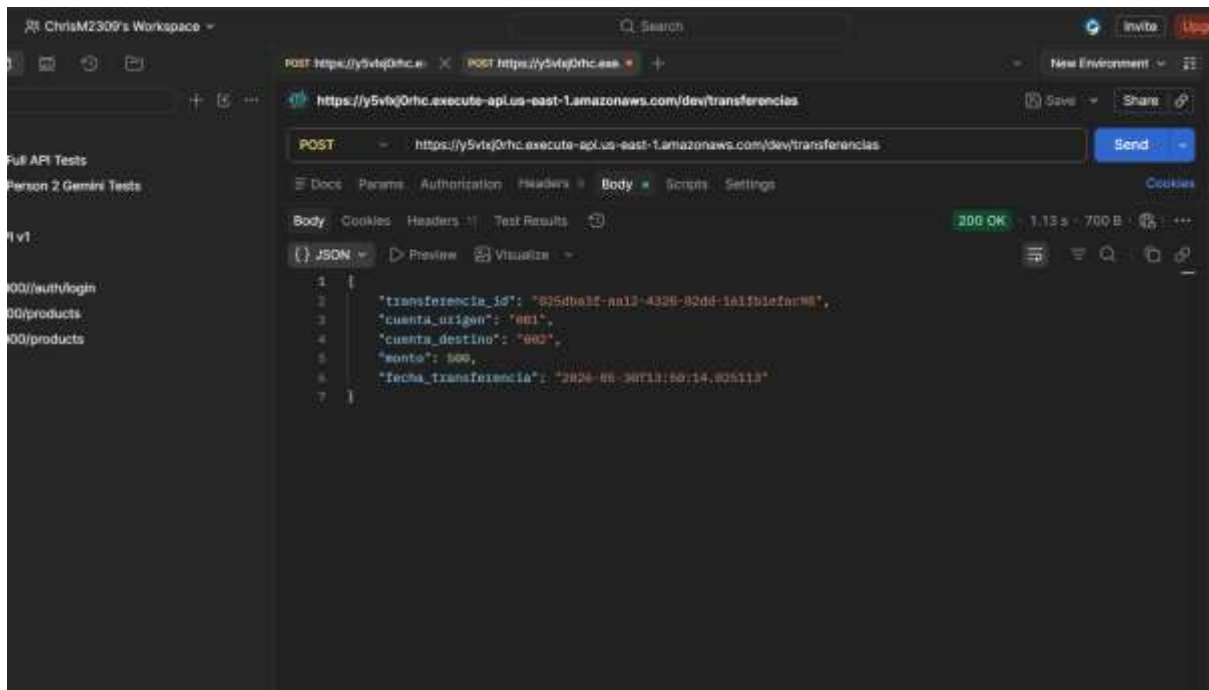
Paso 49

Enviar:

```
{
  "cuenta_origen": "001",
  "cuenta_destino": "002",
  "monto": 500
}
```

Paso 50

Validar respuesta HTTP 200.



Paso 51

Validar transferencia almacenada en DynamoDB.

Parte XIII — CloudWatch Logs

Paso 52

Ingresar a Amazon Web Services CloudWatch.

Paso 53

Abrir:

Logs → Log groups

Paso 54

Seleccionar:

`/aws/lambda/crear_cliente`

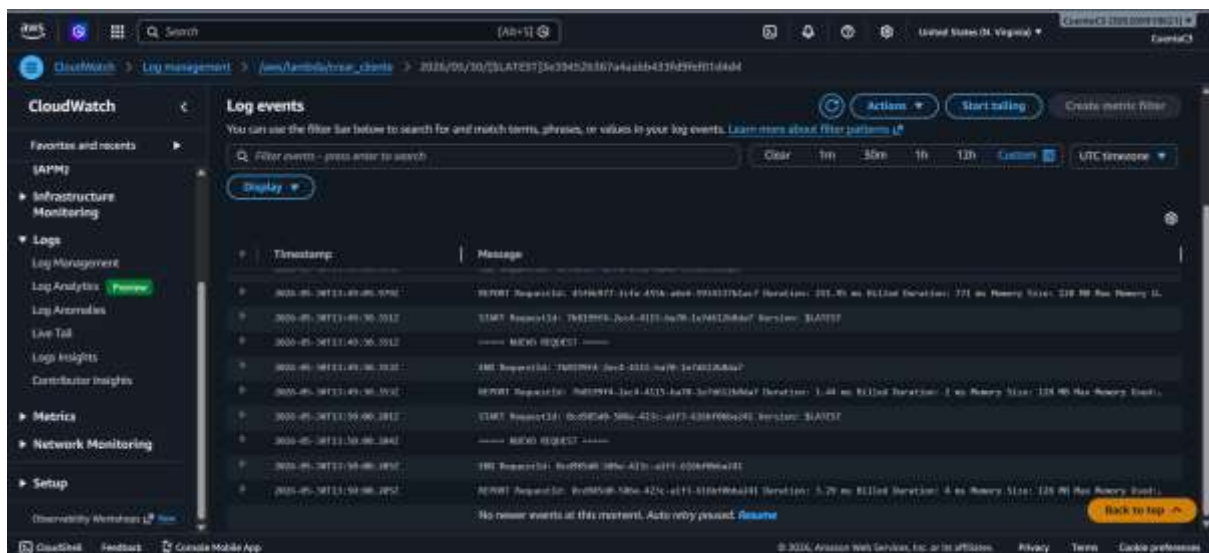
Paso 55

Abrir log stream más reciente.

Paso 56

Validar:

- requests procesados,
- logs personalizados,
- errores,
- clientes creados.



Paso 57

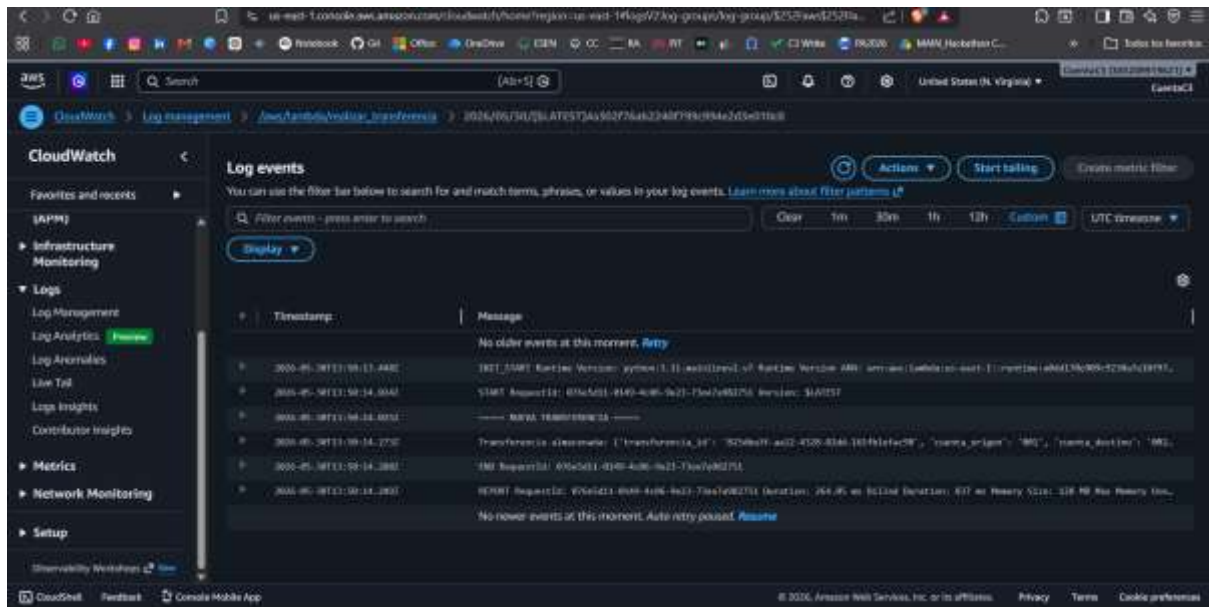
Ingresar a:

/aws/lambda/realizar_transferencia

Paso 58

Validar:

- transferencias procesadas,
- logs,
- requests,
- errores.



Parte XIV — DynamoDB

Paso 59

Ingresar a DynamoDB.

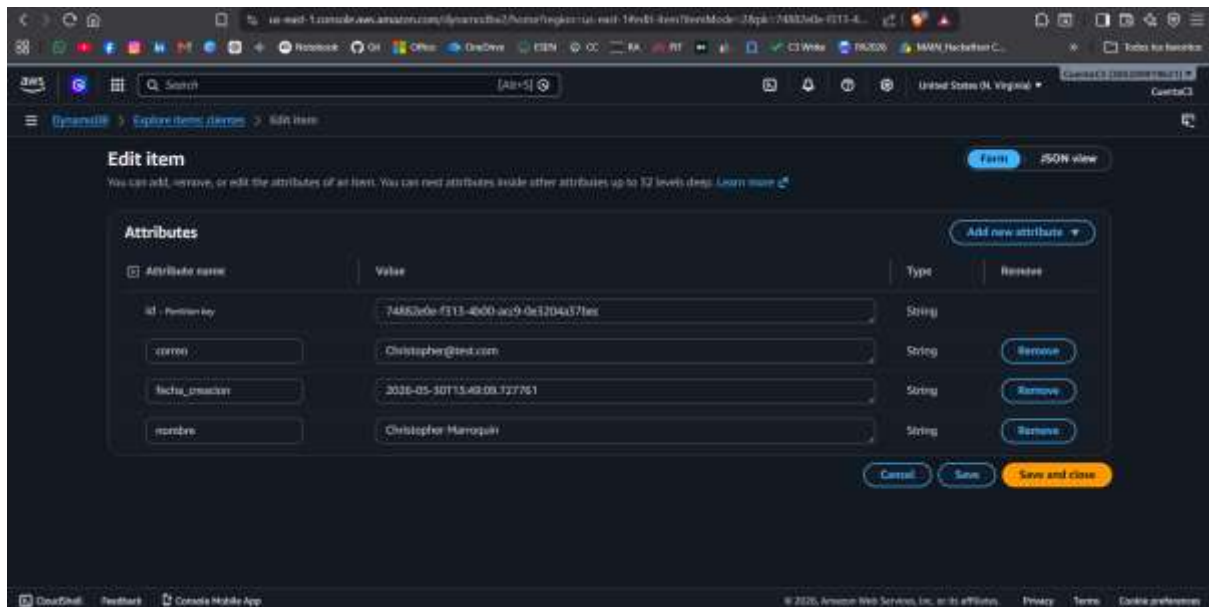
Paso 60

Abrir tabla:

clientes

Paso 61

Validar persistencia de clientes.



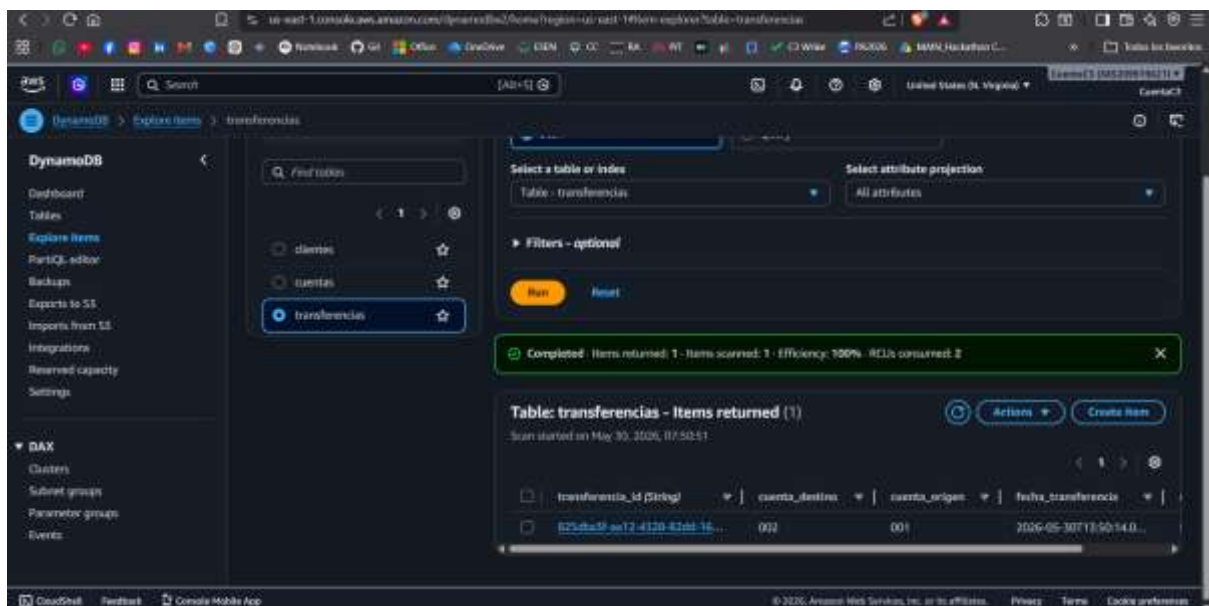
Paso 62

Abrir tabla:

transferecias

Paso 63

Validar persistencia de transferencias.



Parte XV — Terraform y Observabilidad

Paso 64

Modificar cualquier log dentro de una Lambda.

Paso 65

Desde:

terraform/

ejecutar:

terraform fmt

Paso 66

Ejecutar:

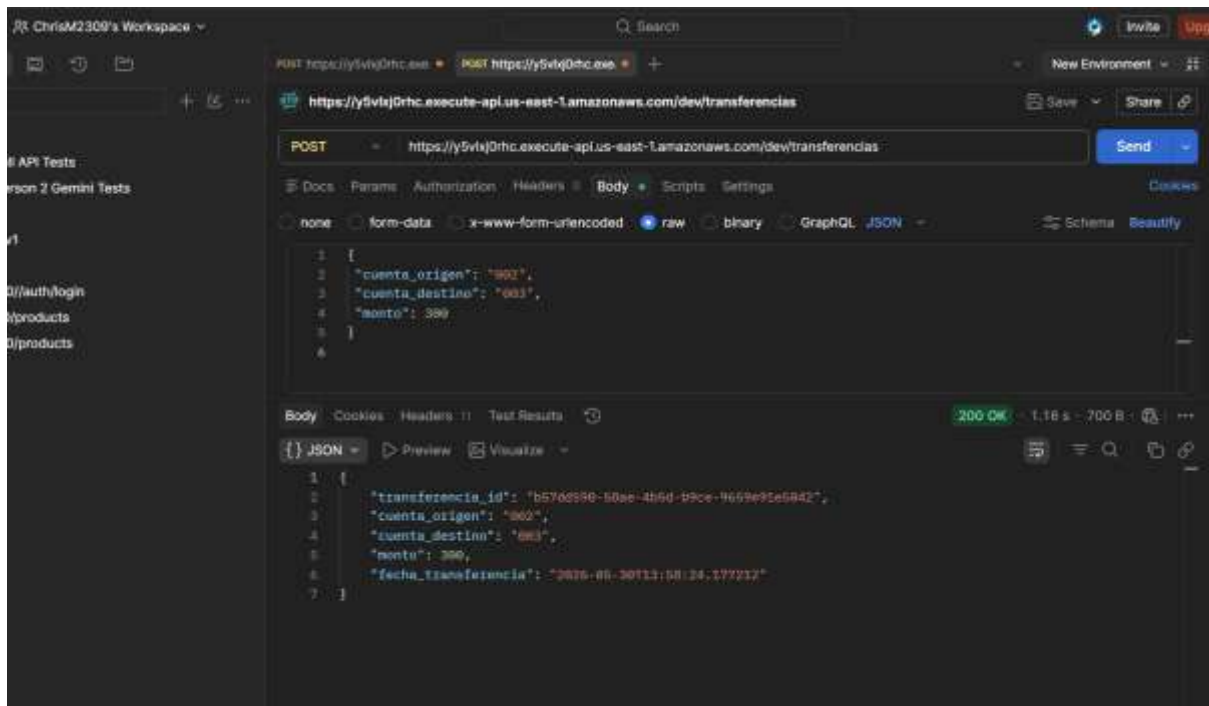
terraform apply

Paso 67

Validar actualización automática de Lambda.

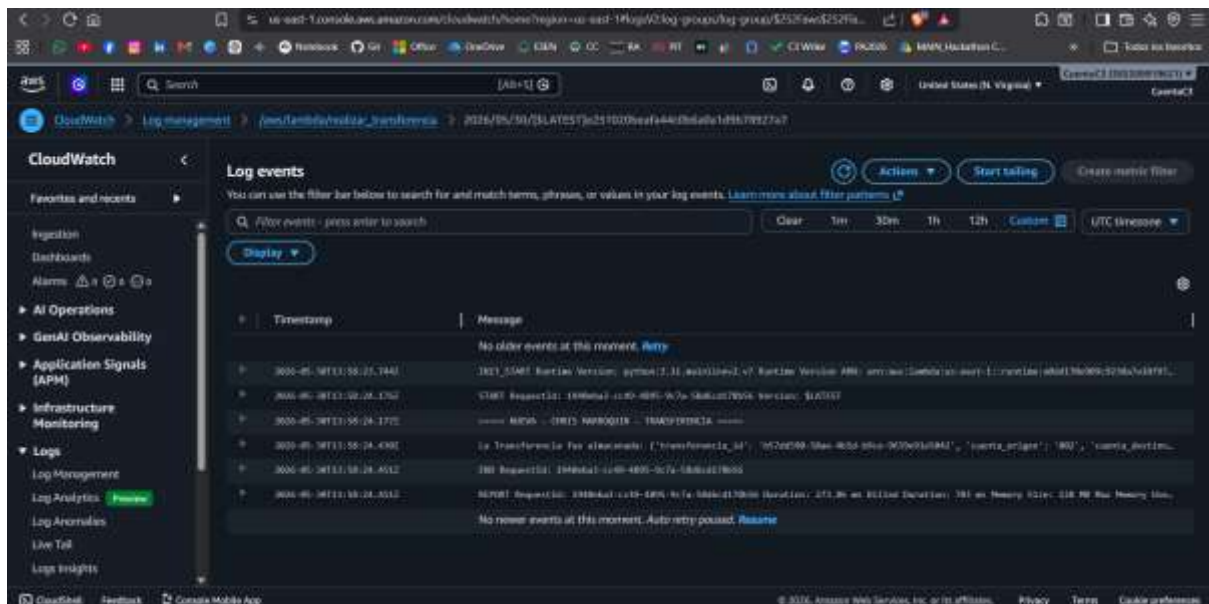
Paso 68

Ejecutar múltiples requests desde Postman.



Paso 69

Validar múltiples ejecuciones en CloudWatch.



Parte XVI — Validación Final

Paso 70

Validar:

- clientes creados,
- transferencias almacenadas,
- logs,
- API Gateway,
- Lambda,
- DynamoDB.

Paso 71

Capturar evidencias finales:

- Terraform,
- CloudWatch,
- DynamoDB,
- Postman,
- estructura proyecto.

Paso 72

Validar que toda la infraestructura fue creada únicamente usando Terraform.

Importante

No crear recursos manualmente desde AWS Console.

No modificar:

- API Gateway,
- DynamoDB,
- IAM,
- Lambda,
- CloudWatch.

Toda infraestructura debe mantenerse administrada únicamente usando Terraform.